Zellic

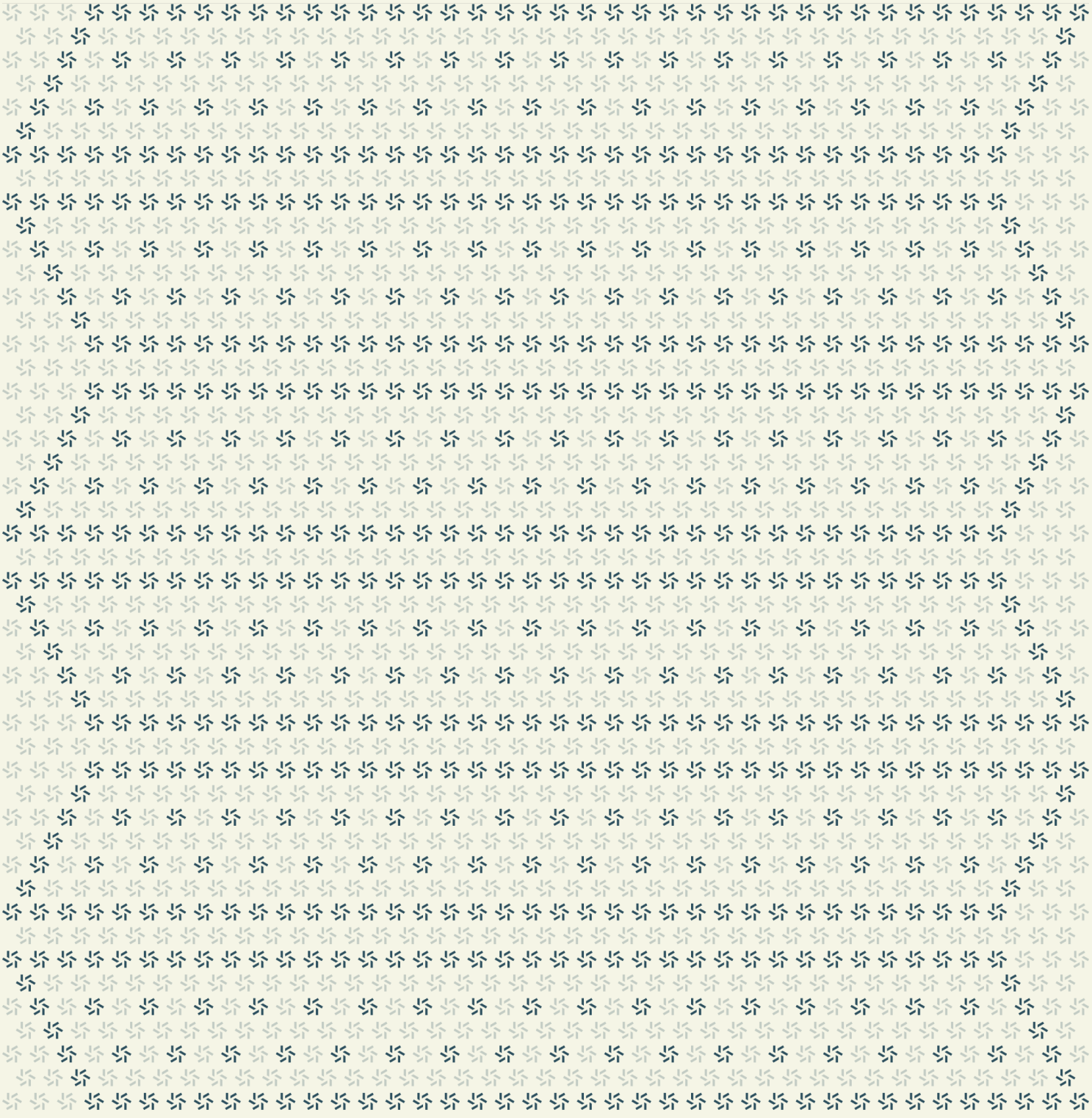
Prepared for
Kelly Zhang
LayerZero

Prepared by
Maik Robert
Guo Xuehao
Zellic

September 9, 2026

Solana Endpoint V2 - Diff

Solana Application Security Assessment



Contents

About Zellic **4**

1. Overview **5**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **7**

2.1. About Solana Endpoint V2 - Diff 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 9

2.5. Project Timeline 10

3. Detailed Findings **11**

3.1. Retained DVN confirmations can re-verify a nilified payload 11

3.2. Stale rent accounting can cause withdrawals from expanded DVN configurations to fail 13

4. Notable Changes **15**

4.1. Major Changes 15

5. Assessment Results **16**

5.1. Disclaimer 16

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for LayerZero on September 4th, 2026. During this engagement, Zellic reviewed Solana Endpoint V2 - Diff's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Do the changes between the specified baseline and target revisions introduce vulnerabilities or break the expected behavior of the in-scope Solana programs and libraries?
- Do the changes preserve authorization, signer, PDA, account-ownership, and cross-program invocation constraints?
- Do the changes preserve correct message-verification and account-lifecycle behavior, including configuration, cleanup, allocation, rent, and fee handling?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

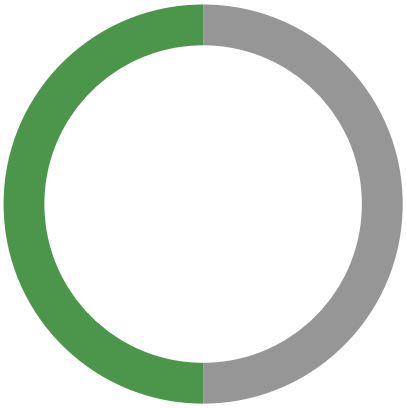
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Solana Endpoint V2 - Diff programs, we discovered two findings. No critical issues were found. One finding was of low impact and the other finding was informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	1
Informational	1



2. Introduction

2.1. About Solana Endpoint V2 - Diff

LayerZero contributed the following description of Solana Endpoint V2 - Diff:

> LayerZero is an immutable, censorship-resistant, and permissionless messaging protocol, that connects blockchains (60+ and counting) to enable omnichain interoperability for blockchain applications. With LayerZero V2, developers have the power to create applications that can seamlessly interact across multiple blockchains.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the programs.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case

basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Solana Endpoint V2 - Diff Programs

Type	Rust
Platform	SVM-compatible
Target	Diff from a177aae in https://github.com/LayerZero-Labs/monorepo to 8842875 in LayerZero-v2
Repository	https://github.com/LayerZero-Labs/LayerZero-v2 ↗
Version	`88428755be6caa71cb1d2926141d73c8989296b5`

Programs	`programs/programs/blocked-messagelib/*`
	`programs/programs/dvn/*`
	`programs/programs/endpoint/*`
	`programs/programs/executor/*`
	`programs/programs/messagelib-interface/*`
	`programs/programs/oft/*`
	`programs/programs/pricefeed/*`
	`programs/programs/uln/*`
	`programs/programs/worker-interface/*`
	`libs/*`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 0.2 person-weeks. The assessment was conducted by two consultants over the course of one calendar day.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Maik Robert
Security Engineer
maik@zellic.io ↗

Guo Xuehao
Security Engineer
guo@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 4, 2026	Start of primary review period
September 4, 2026	End of primary review period

3. Detailed Findings

3.1. Retained DVN confirmations can re-verify a nilified payload

Target	ULN		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

In the baseline implementation, `commit_verification` called the Endpoint and then reclaimed the supplied confirmation accounts in the same transaction:

```
endpoint_verify::verify(
    ctx.accounts.uln.endpoint_program,
    ctx.accounts.uln.key(),
    &params.packet_header,
    params.payload_hash,
    &[ULN_SEED, &[ctx.accounts.uln.bump]],
    &ctx.remaining_accounts[dvns_size..],
)?;

reclaim_storage(confirmation_accounts, ctx.accounts.uln.as_ref())
```

If reclamation failed, the Endpoint verification was rolled back. The target implementation returns after `endpoint_verify::verify` and leaves the confirmation accounts unchanged.

The target moves cleanup to a separate `close_verify` instruction that must be signed by the DVN associated with the confirmation PDA. Because `commit_verification` neither calls nor requires this instruction, confirmations used by a successful commit may remain open. A confirmation account stores its value and bump but has no consumed state or verification generation.

When an authorized receiver or delegate calls `nilify`, the Endpoint replaces the stored payload hash with `NIL_PAYLOAD_HASH`. A permissionless relay can then call `commit_verification` again with the same confirmation accounts. If the accounts remain open, still satisfy the current receive configuration, and ULN remains the active or an unexpired grace-period receive library, the Endpoint restores the original payload hash without a new DVN update.

The confirmation PDAs remain bound to the original packet-header hash, payload hash, and DVNs. This path cannot authenticate a different payload, bypass the current quorum or receive-library checks, or execute the message by itself. It also cannot recreate a payload account closed through `clear`, `burn`, or `skip`. It only restores execution eligibility for the same previously endorsed payload; any duplicate application effect would require a separate downstream path to process that payload again.

Impact

A permissionless relay can undo the temporary execution block established by an authorized `nilify` without obtaining a new DVN confirmation.

Recommendations

Atomically consume the canonical confirmation accounts validated and used by a successful `commit_verification`, or bind confirmations to a verification generation that advances on `nilify`. If cleanup is restored, close only the accounts validated for the successful quorum because optional-threshold evaluation may return before inspecting trailing accounts.

Remediation

LayerZero has acknowledged this issue and provided the following comment:

> Confirmed. This is a known issue. our mitigation is to require DVNs to call `close_verify` as soon as possible after committing. This prevents old confirmations from being reused. We understand that this is an operational mitigation rather than an atomic protocol-level fix.

3.2. Stale rent accounting can cause withdrawals from expanded DVN configurations to fail

Target	DVN		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The target adds `extend_dvn_config`, which expands the canonical DVN configuration from 140 to 200 destination entries. The reallocation reserves `DstConfig::INIT_SPACE` for each of the 60 additional entries, increasing the account's data length by 1,680 bytes. Anchor charges the admin any rent shortfall, so a successful expansion leaves the account rent exempt.

However, `withdraw_fee` still calculates the reserve from the default `DvnConfig::INIT_SPACE`, whose `dst_configs` field is sized for 140 entries:

```
// extend_dvn_config
realloc = 8 + DvnConfig::INIT_SPACE +
    DstConfig::INIT_SPACE * (DST_CONFIG_MAX_LEN - DST_CONFIG_DEFAULT_LEN),

// withdraw_fee
let required_lamports = Rent::get()?.minimum_balance(8 +
    DvnConfig::INIT_SPACE);
let surplus_lamports = ctx.accounts.config.get_lamports() - required_lamports;
require!(surplus_lamports >= params.amount, DvnError::InvalidAmount);
```

This understates the rent required after expansion and overstates the withdrawable surplus. An admin-authorized withdrawal to another account can therefore pass the program's amount check even though it would leave `config` below the rent-exempt minimum for its actual data length. Solana rejects that state transition and rolls back the transfer, so no under-reserved configuration state is committed.

Impact

Some fee withdrawals from expanded DVN configurations fail and must be retried with a smaller amount. No funds are transferred by the rejected transaction, but the transaction fee payer incurs the failed transaction fee.

Recommendations

Calculate the rent reserve from `config`'s current `data_len()` and use checked subtraction when deriving the withdrawable surplus.

Remediation

LayerZero has acknowledged this issue and provided the following comment:

> Confirmed. This is a known issue. Even if `required_lamports` is based on the original `INIT_SPACE`, the Solana runtime should enforce rent-exemption based on the actual account size after `realloc`. So if a withdrawal brings the account below the true minimum balance, the transaction should fail at runtime.

4. Notable Changes

This section highlights notable changes made between reviewed versions.

4.1. Major Changes

- ULN confirmation accounts are no longer reclaimed by `commit_verification`; DVNs can close them separately through `close_verify`.
- Administrators can expand DVN configuration accounts from 140 to 200 destination entries and price-feed accounts from 140 to 340 price entries. DVN destination configurations can also be removed.
- The OApp send helper rejects duplicate sender accounts. Executor's `execute` and `compose` instructions require the executor account to have no data and be owned by the System Program.

5. Assessment Results

During our assessment on the scoped Solana Endpoint V2 - Diff programs, we discovered two findings. No critical issues were found. One finding was of low impact and the other finding was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.